ADET AUTOMATED TEST

ARMENTA, Sean Dylan
BUERGO, Chenie Niña
CANDELARIA, John Benedict
LITA, Angelica
PISPIS, Dan Emanuel

BSIT 3B
May 19, 2026

TITLE: BRaket: A Role-Based Talent Discovery and Commission Platform for BU Students

Member	Assigned Feature	Type of Test	Tool/Framework	Main Output
BUERGO	Login and Authentication + Documentation Lead	Functional + error handling test	Playwright / Cypress / Selenium	Test valid login, invalid login, empty fields, logout/session behavior
PISPIS	Registration / Profile Creation	Input validation test	Playwright / Cypress / Selenium	Test required fields, duplicate email/username, successful account/profile creation
ARMENTA	Service Listings	CRUD test	Playwright / Cypress / Selenium	Test creating, editing, viewing, and deleting/updating service listings
LITA	Search, Filter, and Browse Talents/Services	UI functionality test	Playwright / Cypress / Selenium	Test search keywords, category filters, price filters, and empty search results
CANDELARIA	Booking / Commission Flow	Workflow test	Playwright / Cypress / Selenium + Docs	Test booking request creation/status behavior; compile group journal, screenshots, findings, and final submission

I. **BUERGO** (Login and Authentication, Playwright)

Test Scenario Documentation:

- **Functionality Tested**

  Login and authentication functionality of the BRaket web application.

- **Objective of the Test**

  The objective of this automated test is to verify whether the BRaket login page loads properly and responds correctly to common user actions, such as viewing

the login form, submitting an empty form, entering invalid credentials, and accessing the forgot password option.
The test also aims to check if the system prevents unauthorized access when incorrect login details are entered.

- **Steps / Procedure**
    1. Open the BRaket web application in the browser.
    2. Navigate to the login page.
    3. Verify that the login page displays the heading **“Welcome back.”**
    4. Verify that the login form is visible.
    5. Check if the email input field is displayed.
    6. Click the **Sign in** button without entering login details.
    7. Observe whether the system stays on the login page or shows validation behavior.
    8. Enter invalid login credentials.
    9. Click the **Sign in** button.
    10. Observe whether the user is prevented from accessing the dashboard.
    11. Click the **Forgot password** option.
    12. Observe whether the password recovery option or related interface is accessible.
    13. Run the test using Playwright in Chromium, Firefox, and WebKit browsers.
    14. Review the Playwright test report and record the actual results.

- **Test Data / Input**

Test Case	Test Data / Input
TC-LOGIN-001	Navigate to /login
TC-LOGIN-002	Check if the email field appears
TC-LOGIN-003	Click “Sign in” with empty fields
TC-LOGIN–004	Email: wrong-user@example.com Password: wrongpassword123
TC-LOGIN-005	Click “Forgot password”

- **Expected Result**

Test Case	Expected Result
TC-LOGIN-001	The login page should load successfully and display the heading “Welcome back”
TC-LOGIN-002	The login form should display the email input field
TC-LOGIN-003	The system should not allow login with empty fields. The user should remain on the login page or receive a validation

	response.
TC-LOGIN–004	The system should reject invalid credentials and should not redirect the user to the dashboard.
TC-LOGIN-005	The forgot password option should be accessible and should display a password recovery form, message, or related action.

- **Actual Result**
  Based on the Playwright test report, a total of 21 automated tests were executed. Out of these, 15 tests passed and 6 tests failed.

  The passed tests confirmed that the login page loads correctly, the email field is visible, and the empty login submission does not proceed to another page.

  The failed tests were related to invalid login credential handling and forgot password verification. These failed in Chromium, Firefox, and WebKit, which means the issue was not limited to only one browser.

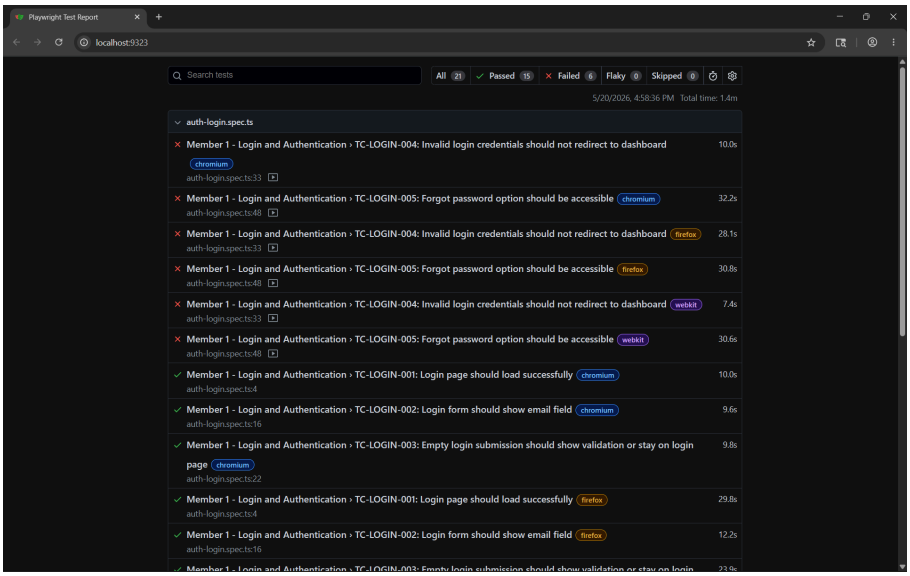
Test Case	Actual Result
TC-LOGIN-001	The login page loaded successfully in the browser.
TC-LOGIN-002	The email input field was visible on the login form.
TC-LOGIN-003	Submitting the login form with empty fields did not redirect the user away from the login page.
TC-LOGIN–004	The test failed because the automated script was not able to clearly confirm the expected invalid login response within the test run.
TC-LOGIN-005	The test failed because the automated script was not able to properly verify the forgot password interface after clicking the option.

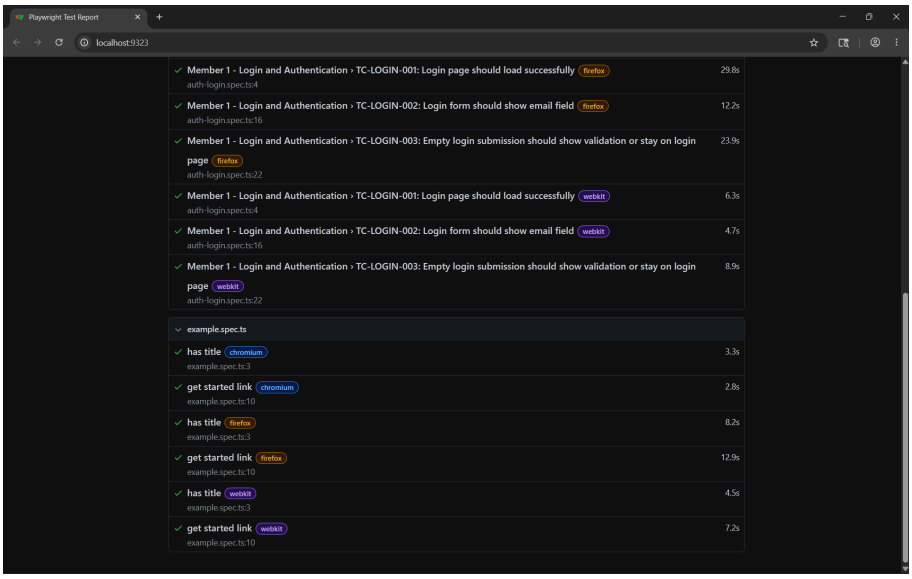
- **Status**

Test Case	Status
TC-LOGIN-001: Login page should load successfully	Passed

TC-LOGIN-002: Login form should show email field	Passed
TC-LOGIN-003: Empty login submission should show validation or stay on login page	Passed
TC-LOGIN-004: Invalid login credentials should not redirect to dashboard	Failed
TC-LOGIN-005: Forgot password option should be accessible	Failed

Overall Status: Partially Passed





Reflection / Findings

- **Issues Encountered**

  During the automated testing of the login and authentication feature, two test scenarios failed across all three browsers tested: Chromium, Firefox, and WebKit.

  The first issue occurred in the invalid login credentials test. The test entered an incorrect email and password, then checked whether the user would be prevented from accessing the dashboard. However, the test failed because the automated script was not able to clearly confirm the expected response after the invalid login attempt.

  The second issue occurred in the forgot password test. The test clicked the forgot password option, but the automated script was not able to properly verify whether the password recovery interface appeared.

  Another issue encountered was that the Playwright report included the default sample test file, example.spec.ts. These sample tests were not related to the BRaket login feature and should be removed from the final test run.

- **Warnings / Errors Detected**

  The Playwright report showed 6 failed tests. These failures came from two test cases running across three browsers.

  The failed test cases were:

  1. TC-LOGIN-004: Invalid login credentials should not redirect to dashboard
  2. TC-LOGIN-005: Forgot password option should be accessible

  Since both tests failed in Chromium, Firefox, and WebKit, the problem is likely related to the test logic or how the login page responds, rather than a browser-specific issue.

  The terminal also showed Git warnings about line endings, such as LF being replaced by CRLF. This is common when working on Windows and does not directly affect the functionality of the automated test.

- **Bugs Discovered**

  No major system-breaking bug was confirmed during the test.

  However, the test results revealed possible areas for improvement:

  1. The invalid login behavior needs a clearer response that can be easily verified by automated testing.
  2. The forgot password flow needs a more specific visible message, heading, or button that confirms the action was successful.
  3. The automated test script needs to check more specific output instead of using broad or unclear conditions.

  These findings may not mean that the login feature itself is broken. Instead, they show that the test script and user interface feedback should be improved to make the results easier to verify.

- **Improvements Made After Testing**
  After reviewing the Playwright report, the following improvements were identified:
  1. The invalid login test should be improved by checking for a clear error message, such as “Invalid credentials” or “Login failed,” instead of only checking that the page does not redirect to the dashboard.
  2. The forgot password test should be improved by checking for a more specific password recovery message, heading, or button, such as “Reset password,” “Send reset link,” or “Check your email.”
  3. The default Playwright sample test file, example.spec.ts, should be removed so that the final test report only includes BRaket-related test cases.
  4. The screenshots and test report should be saved in a dedicated documentation folder so that the testing evidence is organized and easier to review.
  5. Future tests may use a dedicated test account to verify successful login without affecting real user accounts.

- **Lessons Learned from Automation Testing**
  This activity showed that automated testing is useful for checking whether important system features work as expected. It also helps identify unclear behavior that may not be obvious during manual testing.

  Through Playwright, the login page was tested in multiple browsers, which helped confirm whether the results were consistent across different browser environments.

  The activity also showed that automated tests must have clear and specific expected results. If the test only checks broad conditions, such as whether the user stayed on the same page, the result may become unclear. A better test should look for specific messages, buttons, or page changes.

  Another lesson learned is that failed tests are still valuable. A failed test does not always mean the system is completely broken. It can also mean that the system response, test script, or user feedback needs improvement.

  Overall, automated testing helped improve the quality assurance process by providing evidence, screenshots, and reports that can be used to evaluate and improve the BRaket system.

II. **PISPIS** (Registration / Profile Creation, Selenium)

Test Scenario Documentation:

- **Functionality Tested**
  - Registration and Profile Creation for both Client and Talent in the BRaket web application.
- **Objective of the Test**
  - The main objective of the test is to evaluate whether the UI/UX meets design standards, fulfills system requirements, and functions as intended to provide a satisfactory user experience. Specifically, the test verifies input field validation,

duplicate email/username detection, successful account and profile creation, and the system's overall usability, responsiveness, and functionality.

- **Steps and Procedure**

1. Prepare The Application
   - Open the BRaket project folder.
   - Start the main application server: *(npm run dev)*
   - Configure that the application is running in: (localhost)

2. Prepare The Tester
   - Open another terminal
   - Go to tester folder
   - Install tester dependencies if not yet installed *(npm install)*

3. Set Up The Test Environment
   - Copy the sample .env file *(copy .env.example .env)*
   - Edit *(Tester/.env)*
   - Add the following values:
     - *QA_CLIENT_EMAIL*
     - *QA_CLIENT_PASSWORD*
     - *QA_TALENT_EMAIL*
     - *QA_TALENT_PASSWORD*
     - *QA_DUPLICATE_EMAIL*
     - *QA_DUPLICATE_USERNAME*

4. Run The Tester UI
   - Start the tester dashboard: *(npm run ui)*
   - Open the tester page

5. Check Application Status
   - Make sure that the dashboard says that the app is online
   - If it is offline, confirm (npm run dev)
6. Run Registration Tests
   - Click run registration
   - The tester checks:
     - Required registration fields
     - Short password validation
     - Duplicate email rejection
     - Successful new account request

7. Run Profile Tests
   - Click Run Profiles
   - The tester checks:
     - Required client profile fields
     - Duplicate username rejection
     - Successful client profile update
     - Requires talent profile fields
     - Successful talent profile step completion

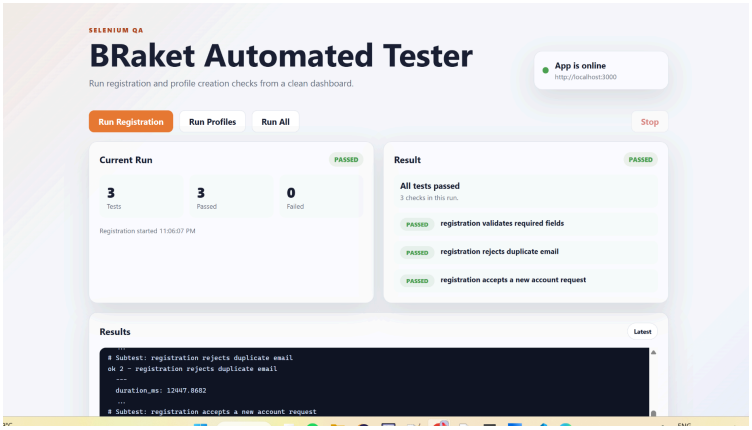
8. Review The Results
    - View the Result panel for a simple summary
    - Passed tests are marked as PASSED
    - Failed tests are marked as FAILED with the main error message
    - The raw CLI output is shown below for debugging purposes

9. Fix Errors If Needed
    - If login fails, check the QA email and password on Tester/.env
    - If the app is offline, restart the main app server
    - If duplicate username testing fails, make sure `QA_DUPLICATE_USERNAME` belongs to another existing account

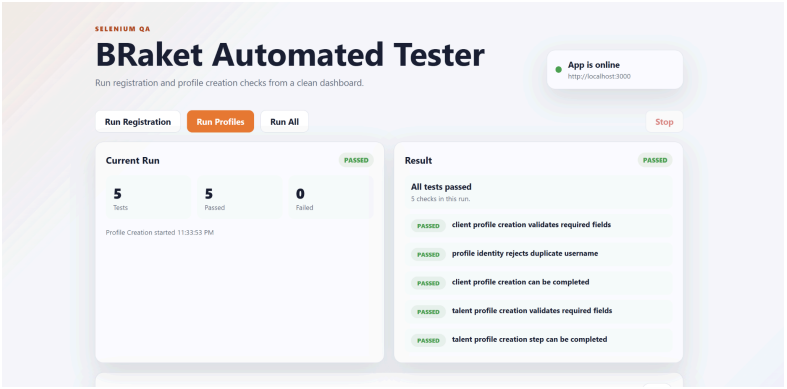
10. Record The Output
    - Save or screenshot the final tester result
    - Include the number of passed and failed test in the report

- **Test Data/Input**
    - New Testing Email
    - Existing Email Account
    - Account password
    - Organization name (BRaket QA Studio)
    - Business Address (Bicol University)
    - Website (localhost)
    - Existing talent and client information
- **Expected Result**
    - Talent/Client Registration
        - Registration validates required fields - *must* PASS
        - Registration rejects duplicate email - *must* PASS
        - Registration accepts a new account request - *must* PASS
    - Talent/Client Profile
        - Client profile creation validates required fields - *must* PASS
        - Profile identity rejects duplicate username - *must* PASS
        - Client profile creation can be completed - *must* PASS
        - Talent profile creation validates request fields - *must* PASS
        - Talent profile creation step can be completed - *must* PASS
- **Actual Result**
    - Talent/Client Registration
        - Registration validates required fields - PASSED
        - Registration rejects duplicate email - PASSED
        - Registration accepts a new account request - PASSED
    - Talent/Client Profile
        - Client profile creation validates required fields - PASSED
        - Profile identity rejects duplicate username - PASSED
        - Client profile creation can be completed - PASSED
        - Talent profile creation validates request fields - PASSED
        - Talent profile creation step can be completed - PASSED
- **Status**
    - Talent/Client Registration **(3/3 - PASSED)**
    - Talent/Client Profile **(5/5 - PASSED)**

- **Screenshots / Evidence**
    - Client/Talent Registration



    - Client/Talent Profile



Reflection and Findings

Creating an automated quality assurance or tester is new to me; I am not aware of how automation will work. That thinking alone held me back for some time before I started to do it.

Entering a new path as an IT student is very exciting. I tried and encountered several hard topics or projects, and I passed them; that's why I am confident that I can make this happen or pass this one. The main highlight of this activity is the issues and results. I encountered several issues, both technical and personal issues. I alone am not that good at creating automations; that's why it's hard for me to make a code for it. I am more of a traditional QA rather than a QA that likes to automate things. One issue I encountered is that I had a hard time accessing the profiles of clients and talent. It's hard because I need to make a way for it to happen; it always gets an error because it cannot access the client and talent profile because it takes too long and it can't enter the website itself. Both client/talent profiles and client/talent registration are connected

to their email; you cannot create or access a profile when you don't confirm it on your email. Next, some warnings or errors I encountered are on the code itself and not on the website; I got errors on running my tester on CLI and had a hard time connecting it to a web-based UI. Some bugs I discovered are that when you open a localhost and also open the tester, the localhost tends to disconnect from the tester, and it causes the tester to become offline. This makes the testing longer and not that accurate.

After several testings and revisions, improvements on data flow and client/talent registration and profile are observed. We made the emails OTP and emails verifier faster to be delivered to their emails to prevent delays and spam on requesting OTP. We also made changes to our database because some are not aligned to our liking and system functionalities.

After conducting this automation testing, I didn't just get a new skill to develop, but I got a chance to look at different perspectives in our system. A kind of testing that removes biases and has guidelines or criteria. Automated testing is very great and exciting to explore.

III. ARMENTA

Test Scenario Documentation

- **Functionality Tested**

  The automated tests cover the Service Listings feature in the talent dashboard.

  Specifically, the tests verify that a talent user can:

  - Create a new service listing
  - View the created service on the My Services page
  - Edit an existing service listing
  - Delete a service listing
  - Prevent invalid price input, such as minimum price greater than maximum price
  - Confirm service changes in the database using Prisma
  - Block unauthorized access from unauthenticated users and client accounts

  Test files used:

  - service-listings-create.spec.ts
  - service-listings-edit.spec.ts
  - service-listings-delete.spec.ts
  - service-listings-validation-security.spec.ts

- **Objective of the Test**

  The objective is to verify that the Service Listings feature works correctly, securely, and consistently through automated Playwright testing.

  The test checks that valid service listing actions, such as creating, editing, and deleting services, are saved and displayed properly. It also ensures that invalid inputs, such as an incorrect price range, are rejected before they affect the database. In addition, the test confirms that only authorized talent users can access and manage service listings.

- **Steps/ Procedure**
    1. Execute the Playwright E2E suite using npm run test:e2e
    2. Initialize test fixtures using Prisma and Supabase Admin API.
    3. Create dedicated E2E accounts for talent and client roles.
    4. Authenticate the talent account through the real login page.
    5. Navigate to /dashboard/talent/services.
    6. Open the create service route /dashboard/talent/services/new.
    7. Fill out the service form using Playwright locators such as getByLabel() and getByRole().
    8. Submit valid service data and assert successful redirect back to /dashboard/talent/services.
    9. Verify the created service in the UI using Playwright expect() assertions.
    10. Verify database persistence using Prisma service queries.
    11. Seed an existing service record for edit and delete test cases.
    12. Open the edit dialog and update service fields.
    13. Assert that edited values are reflected in both the UI and database.
    14. Trigger the delete confirmation dialog and confirm deletion.
    15. Assert that the deleted service is hidden from the UI and removed from the database.
    16. Submit invalid price data where minPrice > maxPrice.
    17. Assert that the validation message appears and no database record is created.
    18. Test access control by visiting the route while unauthenticated.
    19. Test role-based protection by signing in as a client user and attempting to access talent service management.

- **Test Data/ Input**

Test Data	Value / Description
ACCOUNTS	
Talent test account	Generated @mock.braket.local talent user
Client test account	Generated @mock.braket.local client user
Password	ArmentaService123!
SERVICE DETAILS	
Service category	Graphic Design
Price unit	Per project
PRICING	
Valid minimum price	2500
Valid maximum price	6800
Edited maximum price	7200
Invalid minimum price	9000
Invalid maximum price	1000
STACK	
Test framework	Playwright
Database tool	Prisma
Auth setup	Supabase Admin API
ROUTES	
Test route	/dashboard/talent/services
Create route	/dashboard/talent/services/new

- **Expected Result**

  The Service Listings feature should allow an authenticated talent user to successfully create, view, edit, and delete service listings.

  Expected behavior:

  - Valid service data should be accepted.
  - Created services should appear on the My Services page.
  - Created services should be saved in the database.
  - Edited service details should update correctly in the UI.
  - Edited service details should persist in the database.
  - Deleted services should disappear from the UI.
  - Deleted services should be removed from the database.
  - Invalid price ranges, such as minPrice > maxPrice, should be rejected.
  - Invalid service submissions should not create database records.
  - Unauthenticated users should be redirected to the login page.
  - Client accounts should not be allowed to access talent service management.

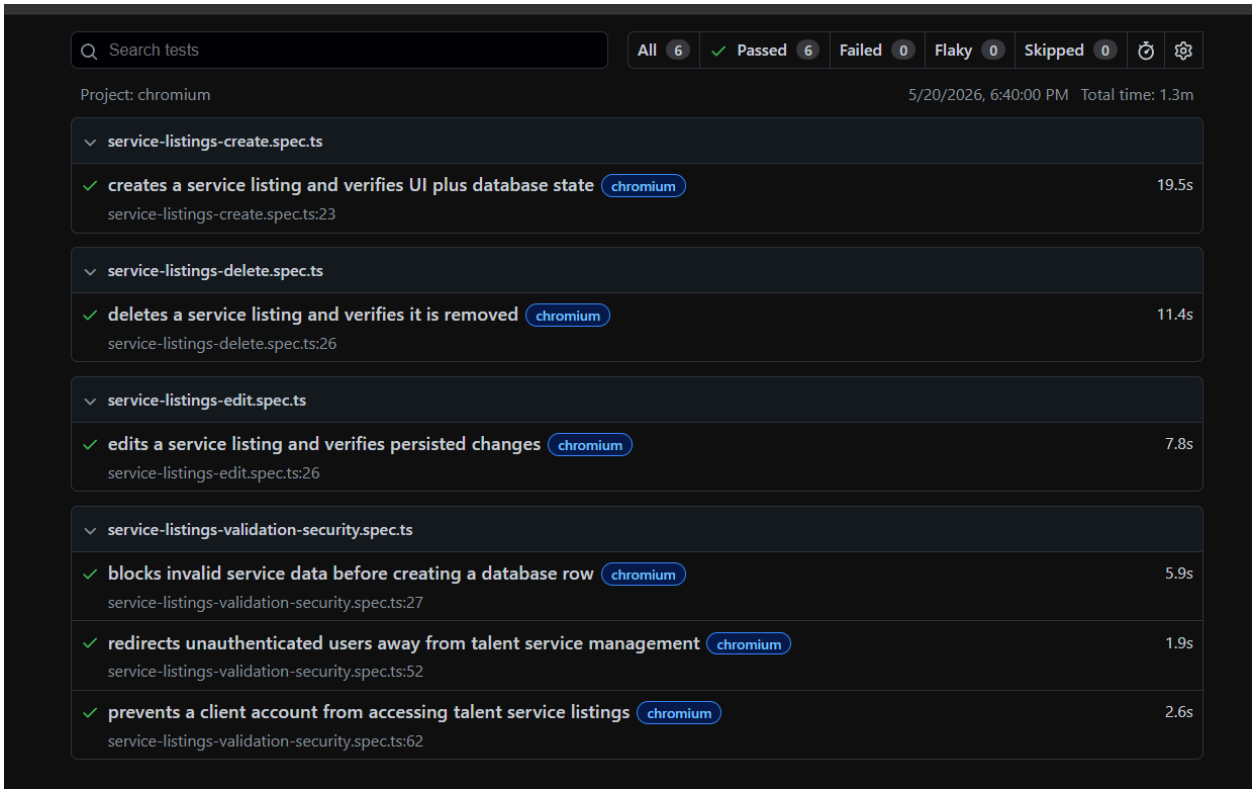
- **Actual Result**

  The Playwright E2E test suite passed successfully.

  Actual results:

- All service listing tests executed successfully.
- Total tests passed: 6
- Total automated test scripts used: 4
- The talent user was able to create a valid service listing.
- The created service appeared correctly on the My Services page.
- The service was confirmed in the database using Prisma.
- Existing services were edited successfully.
- Updated service details were reflected in both the UI and database.
- Deleted services were removed from the UI and database.
- Invalid price input was blocked by validation.
- Unauthenticated users were redirected to login.
- Client users were prevented from accessing talent service management.

- **Status (Passed / Failed)**



Reflection/ Findings

- **Issues encountered**
  1. Several issues were encountered while creating and running the automated tests:
  2. Some Playwright locators were too broad and matched multiple elements.
     Example: getByLabel("Password") matched both the password input and the show-password button.
  3. The sign-in button locator also matched the Google sign-in button.
     This was fixed by using an exact role locator:
     getByRole("button", { exact: true, name: "Sign In" })

4. The service creation test initially stayed on /dashboard/talent/services/new after submission because the test talent profile did not satisfy onboarding finalization rules.
5. The test fixture had to be improved by adding valid talent profile details and skills.
6. The price assertion initially expected a different format.
   The app displayed prices as ₱2,500 - ₱6,800, so the test assertion was updated to match the actual UI output.

- **Warnings / errors detected**

  Password locator conflict

  - Error: getByLabel("Password") matched the input and show-password button.

```
page.locator('input[name="password"]')
```

  Sign In button conflict

  - Error: getByRole("button", { name: "Sign In" }) also matched Google sign-in.

```
page.getByRole("button", { exact: true, name: "Sign In" })
```

  Create service redirect failed

  - Error: Expected /dashboard/talent/services, but stayed on /dashboard/talent/services/new.
  - Cause: Test talent profile was incomplete.
  - Fix: Added valid profile data and required skills in the test fixture.

  Price format mismatch

  - Error: Expected PHP 2,500.00 - PHP 6,800.00
  - Actual UI: ₱2,500 - ₱6,800

```
page.getByText(new RegExp("\\u20b12,500 - \\u20b16,800"))
```

- **Bugs discovered**
  a. Ambiguous login form accessibility labels
     - The password field and show-password button were both detected by getByLabel("Password").
     - This suggests the login form can be improved with clearer accessible labels for automated testing and accessibility.
  b. Service creation depends on complete talent profile data

        - The service creation flow failed when the test talent account had an incomplete talent profile.
        - This shows that service creation is tightly connected to onboarding completion rules.
    c. Price display format mismatch
        - The test expected PHP 2,500.00 - PHP 6,800.00, but the UI displayed ₱2,500 - ₱6,800.
        - This indicates that tests must follow the actual UI formatting, and documentation should match the displayed format.

- **Improvements made after testing**
    - Split the original single test into four focused Playwright test scripts:
        - Create service
        - Edit service
        - Delete service
        - Validation and security
    - Added Prisma database checks to confirm that service changes were actually saved or removed.
    - Improved test fixtures by creating valid Supabase talent and client test accounts.
    - Added validation testing for invalid price ranges.
    - Added access-control testing for unauthenticated users and client accounts.
    - Added a GitHub Actions workflow for CI/CD pipeline testing.
    - Updated the Playwright report setup to provide clearer test evidence.

- **Lessons learned from automation testing**

    During the automated testing of the Service Listings feature, I encountered issues with some Playwright locators. Some locators were too broad, such as getByLabel("Password"), which matched both the password input and the show-password button. The sign-in button locator also matched the Google sign-in button, so I fixed it by using a more specific role locator with an exact name. This follows Playwright's recommended use of locators such as getByRole() and getByLabel() for reliable element selection.

    The tests also detected errors in the service creation flow. At first, the test stayed on /dashboard/talent/services/new instead of redirecting back to the services page because the test talent profile was incomplete. I fixed this by improving the test fixture and adding valid talent profile details and required skills. Another issue was the price format mismatch, where the test expected PHP 2,500.00 - PHP 6,800.00, but the UI displayed ₱2,500 - ₱6,800.

    The main bugs discovered were related to ambiguous accessibility labels, incomplete talent profile requirements, and mismatched price formatting. These findings showed that automated testing does not only check if a feature works, but also reveals issues in UI accessibility, validation, and data consistency.

    After testing, I improved the test structure by separating the original test into four focused Playwright scripts for create, edit, delete, validation, and security testing. I also added Prisma database checks, better Supabase test accounts, invalid price validation, access-control testing, and a GitHub Actions workflow for CI/CD testing. Playwright's built-in reports were also useful because they helped review test results more clearly.

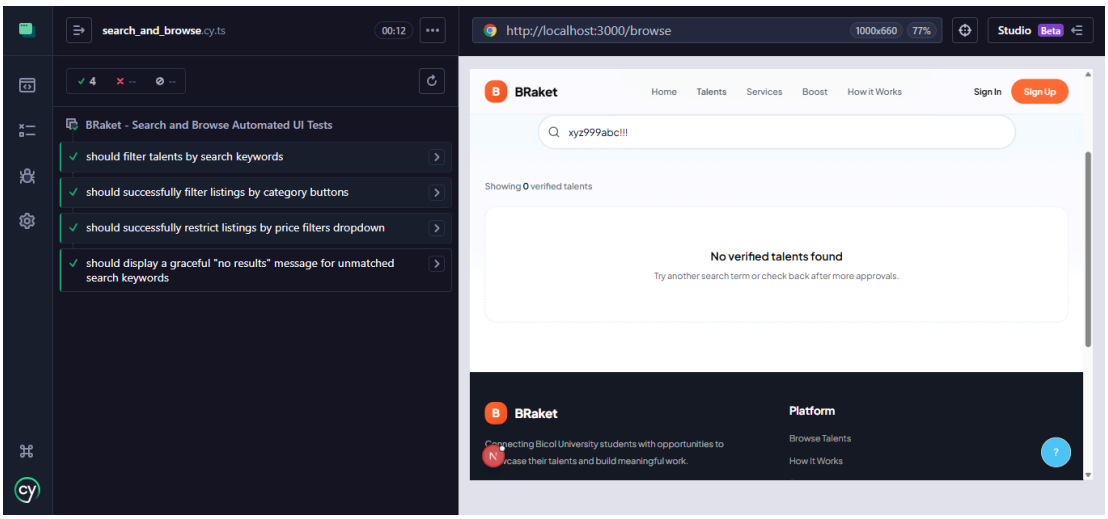
Overall, I learned that automated testing is important because it helps detect errors, unfinished flows, and security issues early. It also makes testing more reliable by confirming both the user interface behavior and the actual database changes.

IV. LITA

- **Functionality Tested:**
  - Search, Filter, and Browse Talents/Services
- **Objective of the Test:**
  - The main objective of the test is to evaluate whether the discovery UI functions as intended to provide a seamless marketplace user experience. Specifically, the test verifies search bar element targeting, interactive category pill response, custom price constraint filter states, and graceful application handling of unmatched empty results.
- **Steps and Procedure:**
  1. **Prepare the Application:** Boot up the local Next.js development environment on http://localhost:3000/.
  2. **Prepare the Tester:** Launch the Cypress Test Suite framework via the project terminal runner.
  3. **Set up the Test Environment:** Verify the automated application lifecycle can successfully hit the target homepage layout.
  4. **Run the Tester UI:** Open the specific spec file to initiate the core script routines.
  5. **Check Application Status:** Ensure cross-page header navigation elements correctly execute routing changes between the /browse and /services components.
  6. **Run Keyword Search Tests:** Execute explicit string injection into the search bar to confirm corresponding data card filtering.
  7. **Run Category Pill Filter Tests:** Programmatically dispatch click action on the category buttons to inspect dynamic layout sorting.
  8. **Run Price Restriction Tests:** Engage the interactive “Max Price” configuration boundary dropdown to validate active view state filtering.
  9. **Run Empty Result Handling Tests:** Submit an unallocated randomized fallback token to confirm the UI displays a clean error-state banner instead of an unhandled script failure.
  10. **Record the Output:** Evaluate runtime assertions, log the four passing test checkmarks, and snap the completion dashboard view for the documentation.
- **Test Data/Input:**
  - Target URLs (`/browse`, `/services`)
  - Valid search keyword string (“Dan Emanuel”)
  - Non-matching, randomized keyword string ("xyz999abc!!!")
  - Target filter selection ("Accounting & Bookkeeping")
  - Target container block ("Max Price:")
  - Existing talent card and service data profiles
- **Expected Results**
  - **Search and Browse Navigation**
    - Navigation links successfully load correct component routes - *must* PASS
  - **Talent / Service Discovery Filter**

        - Keyword search accurately isolates specific matched talent profiles - *must* PASS
        - Category pill button filters visible cards according to selection - *must* PASS
        - Budget selection triggers price parameter fiilter states - *must* PASS
        - Unmatched input strings gracefully render an empty state panel - *must* PASS

- **Actual Result**
    - **Search and Browse Navigation**
        - Navigation links successfully load correct component routes - PASSED
    - **Talent / Service Discovery Filter**
        - Keyword search accurately isolates specific matched talent profiles - PASSED
        - Category pill button filters visible cards according to selection - PASSED
        - Budget selection triggers price parameter filter states - PASSED
        - Unmatched input strings gracefully render an empty state panel - PASSED
- **Status**
    - Search & Browse Navigation (1/1 - PASSED)
    - Talent / Service Discovery Filter (4/4 - PASSED)
- **Screenshots/Evidences:**



Reflection:

This being my very first time encountering automated UI testing, I honestly had no idea where to start and felt completely overwhelmed when my initial scripts kept throwing syntax errors and timing out due to hidden routing mismatches like the `/browse` page path or complex custom buttons. However, by slowing down, analyzing the red error logs on the dashboard, and debugging the element properties step by step, I gradually learned how to properly target selectors and handle dynamic UI states. It slowly started making sense, and it became a highly rewarding experience; my group members and I worked

independently on our respective test files, and their general guidance on the laboratory requirements kept me on the right track.

The pressure and accountability of delivering my part for the team really boosted my perseverance to push through the coding challenges. Ultimately, it was genuinely a satisfying experience to see my work align perfectly with our shared documentation layout and to watch the browser interact with our forms, click filter pills, and handle empty search results entirely on autopilot until we secured all green checkmarks. This activity made me realize there is still so much more for me to discover in IT, as I never previously thought something like automated testing existed for quality checks. I really liked this specific part of our project, and it has definitely opened my eyes to the quality assurance side of development.

V. CANDELARIA

- **Functionality Tested:**
  - Booking / Commission Flow
- **Objective of the Test:**
  - The main objective of this test is to ensure the booking flow works properly. From booking requests down to completing a project, this test shall identify errors and email sending deliverability issues.
- **Steps and Procedure**
  1. I first installed Playwright in the root directory of my local files:
     - npm init playwright@latest
  2. Then I set up playwright.config.ts so the tests could run against the local Next.js app, use Chromium, save test artifacts, and support local test result folders
  3. Next, I created tests\login-two-sessions.spec.ts to verify that I could log in to the two accounts I needed for testing. This confirmed that the client and talent accounts could both be accessed through Playwright's browser context before building the full booking tests.
  4. After that, I wrote tests\booking-request-scenarios.spec.ts. This test covered the booking request form scenarios, including successful submissions with optional fields left blank and blocked submissions when required project details were missing.
  5. I then configured VS Code port forwarding so the Brevo webhook could reach the local development server during testing. This allowed local webhook delivery events to be captured while running the Playwright tests.
  6. Next, I wrote the booking workflow test in tests\booking-status-workflow.spec.ts. This covered the full booking lifecycle: client cancellation, talent decline, talent acceptance, work initiation, work submission, completion approval, and review submission.
  7. To keep the workflow test cleaner, I added shared helpers in tests\booking-workflow-helpers.ts for login, booking card reads, action clicks, status checks, webhook log handling, and email delivery checks.
  8. Then I added tests\booking-review-helpers.ts for the review-specific logic, including getting review context from the database, filling review forms, checking review comments, and waiting for review records to exist.
  9. Finally, I added screenshot capture statements throughout the scripts so each important step produced evidence screenshots before and after key actions.

- **Test Data/Input and Expected Results**

The QA test used two existing accounts to verify separate client and talent sessions through Playwright.

Role	Email	Password	Purpose
Client	cholocandelaria123@gmail.com	Password123!	Sends booking requests, cancels bookings, initiates work, approves completion, and submits client review
Talent	jbbc2023-4132-17458@bicol-u.edu.ph	Password123!	Receives booking requests, declines/accepts bookings, submits work, and submits talent review

Booking Request Test Data

- The booking request scenarios used this service booking page: /book/eca07e1c-493e-4cea-a80e-53def2bf3241

Scenario	Project Details	Budget	Notes	Expected Result
A	Scenario A project details	1500	Scenario A notes	Booking submits successfully and sends email notification to talent
B	Scenario B project details	blank	Scenario B notes	Booking submits successfully and sends email notification to talent
C	Scenario C project details	1800	blank	Booking submits successfully and sends email notification to talent
D	Scenario D project details	blank	blank	Booking submits successfully and sends email notification to talent
E	blank	1900	Scenario E notes	Booking should not submit and no emails will get sent
F	blank	blank	blank	Booking should not submit and no emails will get sent

- For failed submissions, the expected validation message was: **“**Project details are required.”

Booking Status Workflow Test Data

- The booking workflow test used the latest pending bookings from the client booking dashboard: /dashboard/client/bookings
- At least three PENDING bookings were required before running the workflow test:

Booking Used	Action Tested	Expected Status

Latest pending booking	Client cancels booking	CANCELLED and sends an email notification to talent
Next latest pending booking	Talent declines booking	DECLINED and sends an email notification to talent
Next latest pending booking	Talent accepts, work starts, work is submitted, then client approves completion	COMPLETED and send an email notification to talent

Review Feature Test Data

- After completing the accepted booking, both users submitted reviews.

Reviewer	Review Target	Rating	Comment
Client	Talent	5	Client review from the completed booking flow
Talent	Client	4	Talent review from the completed booking flow

- The test also verified that the submitted reviews appeared in the booking cards and public profile/service review sections.

Webhook Test Data

- Brevo email delivery was verified through the local webhook log: test-results/brevo-webhook-events.log
- The webhook checks expected delivered events with booking-related tags such as:

  booking-request:<bookingId>

  booking-cancelled:<bookingId>

  booking-response:<bookingId>:ACCEPTED

  booking-response:<bookingId>:DECLINED

  booking-work-initiated:<bookingId>

  booking-work-submitted:<bookingId>

  booking-work-completed:<bookingId>

VS Code port forwarding was used so Brevo could send webhook events to the local development server during QA testing.

Actual Results

Booking Request

- Booking request with project details, budget, and notes submits successfully - PASSED
- Booking request with empty budget submits successfully - PASSED
- Booking request with empty notes submits successfully - PASSED
- Booking request with empty budget and notes submits successfully - PASSED
- Booking request with empty project details is blocked - PASSED
- Booking request with all fields empty is blocked - PASSED

Booking Status Workflow

- Client can cancel the latest pending booking - PASSED
- Talent can decline the next pending booking - PASSED
- Talent can accept a pending booking - PASSED
- Client can initiate work on an accepted booking - PASSED
- Talent can submit completed work - PASSED
- Client can approve and complete the booking - PASSED

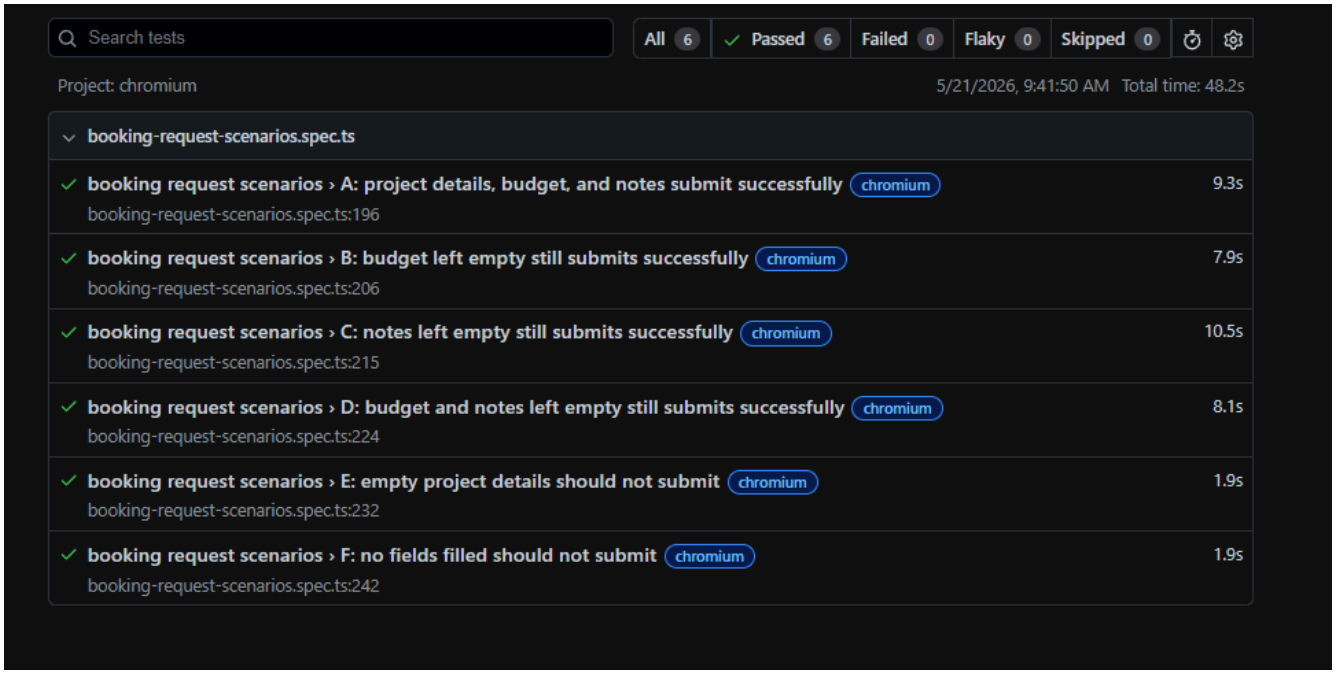
Booking Reviews

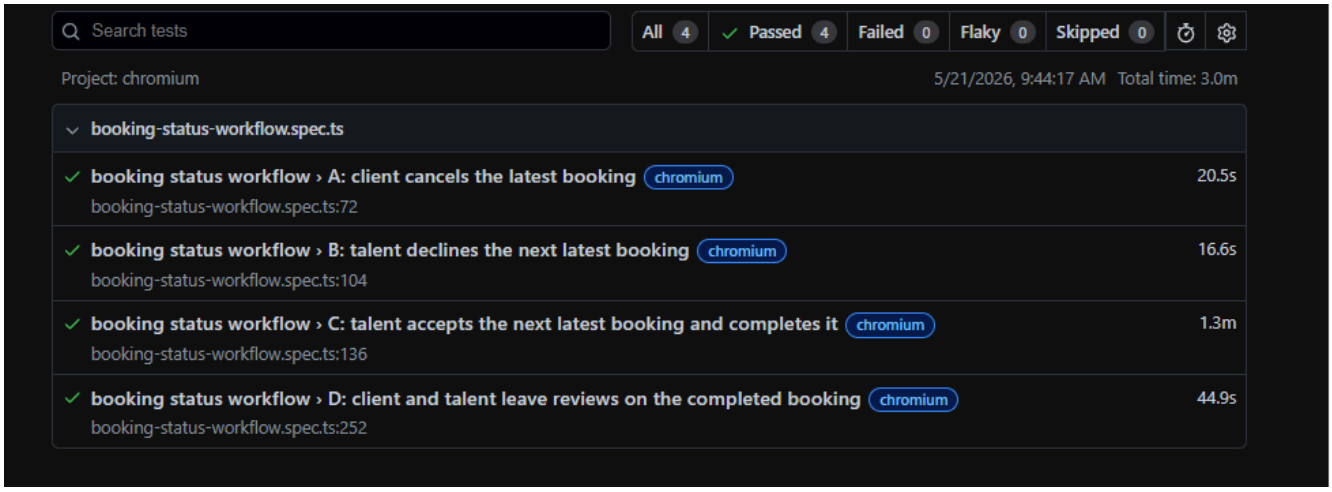
- Client can submit a review for the talent - PASSED
- Talent review appears on the booking record - PASSED
- Talent review appears on the public talent profile - PASSED
- Talent review appears on the service booking page - PASSED
- Talent can submit a review for the client - PASSED
- Client review appears on the booking record - PASSED
- Client review appears on the public client profile - PASSED

Email Webhook Verification

- Booking request email delivery is recorded - PASSED
- Booking cancelled email delivery is recorded - PASSED
- Booking declined email delivery is recorded - PASSED
- Booking accepted email delivery is recorded - PASSED
- Work initiated email delivery is recorded - PASSED
- Work submitted email delivery is recorded - PASSED
- Work completed email delivery is recorded - PASSED

Status: All Passed





Reflection and Findings

During automated testing, one of the main issues I encountered was that Playwright saved screenshots, videos, and other artifacts to a single output folder. Since each new test run could overwrite previous results, I initially thought my past results were not being saved. I also had an issue where videos were not generating properly because my script used two browsers to simulate both the client and talent sides. I later learned that Playwright saves videos only after the browser context is closed, so each context must be handled properly.

I also encountered syntax and logic errors while writing the tests. One issue was that some pages were not paginated, resulting in very long screenshots. At first, I thought Playwright combined multiple screenshots into a single image, but I realized it was capturing the entire page.

The tests also helped reveal unfinished features, especially some email actions that were not yet integrated. Because of this, I added the needed email actions so the test flow could continue properly instead of marking the feature as passed without actual verification.

Overall, I learned that automated testing is important because it helps detect errors, bugs, and incomplete features early. It also makes QA work more efficient by reducing repeated manual testing and providing useful reports, screenshots, and videos for documentation.